SOC Lab Bericht

SSH Brute-Force Detection mit Python & Wazuh

MITRE ATT&CK: T1110.001 – Brute Force: Password Guessing

1. Executive Summary

Dieses Dokument beschreibt den Aufbau und die Durchführung eines Heimlabors zur Simulation und Erkennung eines SSH-Brute-Force-Angriffs. Ziel war es, den Angriff mithilfe eines selbst geschriebenen Python-Skripts realitätsnah nachzubilden und mit dem Open-Source-SIEM Wazuh automatisch zu detektieren.

Alle Phasen wurden vollständig dokumentiert: von der Erstellung der Wazuh-Erkennungsregel über die Angriffssimulation via Python bis hin zu den erzeugten Alerts und der parallelen Netzwerkanalyse mit Wireshark.

2. Laborumgebung

Komponente	Details
Angreifer-System	Windows 11 (DESKTOP-N9PPUE0) – 192.168.222.1
Ziel-System	Ubuntu 24.04
Wazuh Manager	Linux-basiert
Angriffstool	Python 3 mit paramiko – geschriebenes Brute-Force-Skript
Netzwerkanalyse	Wireshark 4.6.4
Erkennungsregel	Wazuh Rule ID 100300 – selbst erstellt

3. Verwendete Tools – technische Erklärung

3.1 Python / paramiko

Paramiko ist eine Python-Bibliothek, die das SSH-Protokoll auf Anwendungsebene implementiert. Sie ermöglicht es, SSH-Verbindungen programmatisch aufzubauen. Das selbst geschriebene Skript sendet in einer Schleife Verbindungsversuche mit falschen Passwörtern an den Zielserver. Jeder Versuch öffnet eine eigene TCP-Session, die nach dem fehlgeschlagenen Authentifizierungsversuch wieder geschlossen wird.

Relevante Parameter im Skript:

- TARGET_IP = 192.168.222.135 – IP des Zielservers
- USERNAME = luan – angegriffener Benutzer
- ATTEMPTS = 10 – Anzahl der Versuche
- DELAY = 0.5s – Pause zwischen den Versuchen
- PASSWORDS = [wrongpassword_0 … wrongpassword_9] – garantiert falsche Passwörter

3.2 Wazuh

Wazuh ist eine Open-Source-SIEM-Plattform. Sie sammelt Logs von Agenten, korreliert Ereignisse anhand von Regelwerken und erzeugt Alerts. Der wazuh-logcollector

überwacht kontinuierlich /var/log/auth.log – sobald eine neue Zeile erscheint, wird sie an wazuh weitergeleitet. Der Decoder extrahiert strukturierte Felder und die Regelmaschine prüft ob eine Regel zutrifft.

3.3 Wireshark

Wireshark ist ein Netzwerk-Protokollanalysator der Pakete in Echtzeit auf einem Netzwerkinterface aufzeichnet. In diesem Lab wurde das VMware VMnet8 Interface überwacht. Wireshark zeigt die Netzwerkebene des Angriffs: die schnellen aufeinanderfolgenden TCP-Verbindungsaufbauten auf Port 22, den SSH-Handshake und die Verbindungsabbrüche nach jedem fehlgeschlagenen Versuch.

4. Regelkonfiguration – Schritt für Schritt

Schritt 1: Eigene Regel erstellen

Die Datei /var/ossec/etc/rules/local_rules.xml wurde mit nano bearbeitet und die Regel 100300 eingetragen. Der Screenshot zeigt die fertige Konfiguration im Editor.

```
luan@wazuh-manager: ~
  GNU nano 7.2                    /var/ossec/etc/rules/local_rules.xml
<!-- Local rules -->

<!-- Modify it at your will. -->
<!-- Copyright (C) 2015, Wazuh Inc. -->

<!-- Example -->
<group name="local,syslog,sshd,">

  <!--
  Dec 10 01:02:02 host sshd[1234]: Failed none for root from 1.1.1.1 port 1066 ssh2
  -->
  <rule id="100001" level="5">
    <if_sid>5716</if_sid>
    <srcip>1.1.1.1</srcip>
    <description>sshd: authentication failed from IP 1.1.1.1.</description>
    <group>authentication_failed,pci_dss_10.2.4,pci_dss_10.2.5,</group>
  </rule>

</group>

<group name="ssh,bruteforce,">

  <rule id="100300" level="10" frequency="5" timeframe="5">
    <if_matched_sid>5760</if_matched_sid>
    <same_source_ip />
    <description>SSH Brute Force detected: 5 failed logins in 5 seconds</description>
    <group>authentication_failures,bruteforce,</group>
  </rule>

</group>

                                  [ Read 30 lines ]
^G Help      ^O Write Out   ^W Where Is    ^K Cut       ^T Execute    ^C Location   M-U Undo
^X Exit      ^R Read File   ^\ Replace     ^U Paste     ^J Justify    ^/ Go To Line M-E Redo
```

Abb. 1 – Wazuh local_rules.xml mit der eigenen Brute-Force-Erkennungsregel 100300

Schritt 2: Konfiguration validieren und Dienst neu starten

```
sudo xmllint --noout /var/ossec/etc/rules/local_rules.xml
sudo systemctl restart wazuh-manager
sudo systemctl status wazuh-manager
```

5. Angriffssimulation – Schritt für Schritt

Schritt 1: Python-Skript starten

Das Skript wurde auf dem Windows-Rechner in PowerShell ausgeführt. Es sendet 10 SSH-Verbindungsversuche mit garantiert falschen Passwörtern (wrongpassword_0 bis wrongpassword_9) an den Zielserver. Zwischen den Versuchen wartet das Skript 0.5 Sekunden.

Schritt 2: Linux empfängt die Verbindungen

Auf dem Zielserver schreiben zwei Komponenten gleichzeitig in /var/log/auth.log: der SSH-Daemon meldet jeden fehlgeschlagenen Passwortversuch, PAM meldet die darunterliegende Authentifizierungsablehnung. Das erzeugt pro Versuch zwei Log-Einträge und damit zwei Wazuh-Alerts.

```
sshd[38985]: Failed password for luan from 192.168.222.1 port 51004
ssh2
sshd[38985]: pam_unix(sshd:auth): authentication failure;
rhost=192.168.222.1 user=luan
```

Schritt 3: Wazuh erkennt das Muster

Wazuh liest /var/log/auth.log kontinuierlich. Der Decoder extrahiert Quell-IP, Port und Benutzer aus jeder Zeile. Die Regelmaschine zählt die Ereignisse pro IP im Zeitfenster. Nach dem fünften Fehlversuch innerhalb von 5 Sekunden feuert Regel 100300.

Alert-Feld	Wert
Rule ID	100300
Level	10 (kritisch)
Beschreibung	SSH Brute Force detected: 5 failed logins in 5 seconds
Src IP	192.168.222.1
User	luan
Quelle	/var/log/auth.log

```
** Alert 1776762391.2383506: - syslog,sshd,authentication_failed,gdpr_IV_35.7.d,gdpr_IV_32.2,gpg13_7.1,hipaa_164.312.b,nist_800_53_AU.14,nist_800_53_AC.7,pci_dss_
10.2.4,pci_dss_10.2.5,tsc_CC6.1,tsc_CC6.8,tsc_CC7.2,tsc_CC7.3,
2026 Apr 21 09:06:31 wazuh-manager->/var/log/auth.log
Rule: 5760 (level 5) -> 'sshd: authentication failed.'
Src IP: 192.168.222.1
Src Port: 51004
User: luan
2026-04-21T09:06:31.209684+00:00 wazuh-manager sshd[38985]: Failed password for luan from 192.168.222.1 port 51004 ssh2

** Alert 1776762391.2384010: - syslog,sshd,authentication_failed,gdpr_IV_35.7.d,gdpr_IV_32.2,gpg13_7.1,hipaa_164.312.b,nist_800_53_AU.14,nist_800_53_AC.7,pci_dss_
10.2.4,pci_dss_10.2.5,tsc_CC6.1,tsc_CC6.8,tsc_CC7.2,tsc_CC7.3,
2026 Apr 21 09:06:31 wazuh-manager->journald
Rule: 5760 (level 5) -> 'sshd: authentication failed.'
Src IP: 192.168.222.1
Src Port: 51004
User: luan
Apr 21 09:06:31 wazuh-manager sshd[38985]: Failed password for luan from 192.168.222.1 port 51004 ssh2

** Alert 1776762394.2384488: - pam,syslog,authentication_failed,pci_dss_10.2.4,pci_dss_10.2.5,gpg13_7.8,gdpr_IV_35.7.d,gdpr_IV_32.2,hipaa_164.312.b,nist_800_53
_AU.14,nist_800_53_AC.7,tsc_CC6.1,tsc_CC6.8,tsc_CC7.2,tsc_CC7.3,
2026 Apr 21 09:06:34 wazuh-manager->/var/log/auth.log
Rule: 5503 (level 5) -> 'PAM: User login failed.'
Src IP: 192.168.222.1
User: luan
2026-04-21T09:06:33.577475+00:00 wazuh-manager sshd[38987]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=192.168.222.1
user=luan
uid: 0
euid: 0
tty: ssh
```

Abb. 2 – Wazuh alerts.log

6. Alert-Analyse

6.1 Regelkette

Jeder Verbindungsversuch erzeugt eine Kette von drei Wazuh-Alerts, die aufeinander aufbauen:

Regel	Bedeutung

5503 Level 5	PAM: User login failed – PAM hat die Authentifizierung abgelehnt
5760 Level 5	sshd: authentication failed – SSH meldet das fehlgeschlagene Passwort
100300 Level 10	SSH Brute Force detected – eigene Regel, feuert nach 5× in 5 Sek.

6.2 Doppelte Alerts – journald und auth.log

Wazuh las in dieser Konfiguration sowohl /var/log/auth.log als auch journald. Da Ubuntu jeden Log-Eintrag in beide Systeme schreibt, erscheint jeder Versuch doppelt in den Alerts – einmal mit Quelle auth.log und einmal mit Quelle journald. Das ist kein Fehler, sondern eine Konfigurationsfrage. In einer produktiven Umgebung würde man eine der beiden Quellen deaktivieren.

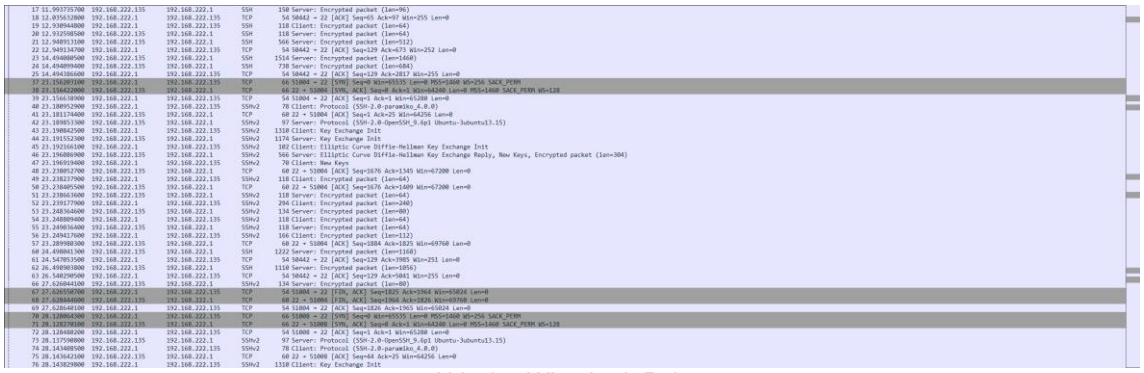
6.3 Port-Muster

Wireshark und die Wazuh-Alerts zeigen übereinstimmend die Ports des Angreifers: 51004, 51008, 51010, 51012, 51014. Windows vergibt für jede neue TCP-Verbindung automatisch einen freien kurzlebigen Port.

6.4 Wireshark-Analyse

Der Screenshot zeigt den Brute-Force-Angriff auf Netzwerkebene. Jeder Passwortversuch des Python-Skripts ist als eigenständige TCP-Session erkennbar: Session 1 auf Port 51004 (Pakete 37–68) und Session 2 auf Port 51008 (ab Paket 70).

Jede Session folgt demselben Muster: TCP-Handshake, SSH-Protokollaustausch, Key Exchange, verschlüsselte Authentifizierung und schließlich FIN sobald das Passwort abgelehnt wurde. Der Übergang von Paket 68 zu 70 zeigt das typische Brute-Force-Muster: kaum ist eine Session mit FIN beendet, startet das Skript sofort die nächste auf einem neuen Port.



17	11.993735700	192.168.222.135	192.168.222.1	SSH	158 Server: Encrypted packet (len=96)
18	12.035652800	192.168.222.1	192.168.222.135	TCP	54 50442 → 22 [ACK] Seq=65 Ack=97 Win=255 Len=0
19	12.930944800	192.168.222.1	192.168.222.135	SSH	118 Client: Encrypted packet (len=64)
20	12.932598500	192.168.222.135	192.168.222.1	SSH	118 Server: Encrypted packet (len=64)
21	12.948913100	192.168.222.135	192.168.222.1	SSH	566 Server: Encrypted packet (len=512)
22	12.948134700	192.168.222.1	192.168.222.135	TCP	54 50442 → 22 [ACK] Seq=129 Ack=673 Win=252 Len=0
23	14.494808500	192.168.222.135	192.168.222.1	SSH	1514 Server: Encrypted packet (len=1460)
24	14.494899400	192.168.222.135	192.168.222.1	SSH	738 Server: Encrypted packet (len=684)
25	14.494386600	192.168.222.1	192.168.222.135	TCP	54 50442 → 22 [ACK] Seq=129 Ack=2017 Win=255 Len=0
37	23.156207100	192.168.222.1	192.168.222.135	TCP	66 51004 → 22 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
38	23.156422000	192.168.222.135	192.168.222.1	TCP	66 22 → 51004 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460 SACK_PERM WS=128
39	23.156618900	192.168.222.1	192.168.222.135	TCP	54 51004 → 22 [ACK] Seq=1 Ack=1 Win=65280 Len=0
40	23.180952900	192.168.222.135	192.168.222.1	SSHv2	78 Client: Protocol (SSH-2.0-paramiko_4.0.0)
41	23.181174400	192.168.222.135	192.168.222.1	TCP	60 22 → 51004 [ACK] Seq=1 Ack=25 Win=64256 Len=0
42	23.183853300	192.168.222.135	192.168.222.1	SSHv2	97 Server: Protocol (SSH-2.0-OpenSSH_9.6p1 Ubuntu-3ubuntu13.15)
43	23.190842500	192.168.222.135	192.168.222.1	SSHv2	1310 Client: Key Exchange Init
44	23.191552300	192.168.222.135	192.168.222.1	SSHv2	1174 Server: Key Exchange Init
45	23.192166100	192.168.222.1	192.168.222.135	SSHv2	102 Client: Elliptic Curve Diffie-Hellman Key Exchange Init
46	23.196086900	192.168.222.135	192.168.222.1	SSHv2	566 Server: Elliptic Curve Diffie-Hellman Key Exchange Reply, New Keys, Encrypted packet (len=304)
47	23.196959400	192.168.222.1	192.168.222.135	SSHv2	70 Client: New Keys
48	23.230052700	192.168.222.135	192.168.222.1	TCP	60 22 → 51004 [ACK] Seq=1676 Ack=1345 Win=67200 Len=0
49	23.230237900	192.168.222.1	192.168.222.135	SSHv2	118 Client: Encrypted packet (len=64)
50	23.230405500	192.168.222.135	192.168.222.1	TCP	60 22 → 51004 [ACK] Seq=1676 Ack=1409 Win=67200 Len=0
51	23.231661600	192.168.222.135	192.168.222.1	SSHv2	118 Server: Encrypted packet (len=64)
52	23.231177900	192.168.222.1	192.168.222.135	SSHv2	294 Client: Encrypted packet (len=240)
53	23.248364600	192.168.222.135	192.168.222.1	SSHv2	134 Server: Encrypted packet (len=80)
54	23.248809400	192.168.222.1	192.168.222.135	SSHv2	118 Client: Encrypted packet (len=64)
55	23.249036400	192.168.222.135	192.168.222.1	SSHv2	118 Server: Encrypted packet (len=64)
56	23.249417600	192.168.222.1	192.168.222.135	SSHv2	166 Client: Encrypted packet (len=112)
57	23.289988300	192.168.222.135	192.168.222.1	TCP	60 22 → 51004 [ACK] Seq=1884 Ack=1825 Win=69760 Len=0
60	24.498041500	192.168.222.135	192.168.222.1	SSH	1222 Server: Encrypted packet (len=1168)
61	24.547015600	192.168.222.1	192.168.222.135	TCP	54 50442 → 22 [ACK] Seq=129 Ack=3985 Win=251 Len=0
62	26.498903800	192.168.222.135	192.168.222.1	SSH	1110 Server: Encrypted packet (len=1056)
63	26.548290500	192.168.222.1	192.168.222.135	TCP	54 50442 → 22 [ACK] Seq=129 Ack=5041 Win=255 Len=0
66	27.626044100	192.168.222.135	192.168.222.1	SSHv2	134 Server: Encrypted packet (len=80)
67	27.626550700	192.168.222.1	192.168.222.135	TCP	54 51004 → 22 [FIN, ACK] Seq=1825 Ack=1964 Win=65024 Len=0
68	27.628444600	192.168.222.135	192.168.222.1	TCP	60 22 → 51004 [FIN, ACK] Seq=1964 Ack=1826 Win=69760 Len=0
69	27.628640100	192.168.222.1	192.168.222.135	TCP	54 51004 → 22 [ACK] Seq=1826 Ack=1965 Win=65024 Len=0
70	28.120054200	192.168.222.1	192.168.222.135	TCP	66 51008 → 22 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
71	28.120278100	192.168.222.135	192.168.222.1	TCP	66 22 → 51008 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460 SACK_PERM WS=128
72	28.120488200	192.168.222.1	192.168.222.135	TCP	54 51008 → 22 [ACK] Seq=1 Ack=1 Win=65280 Len=0
73	28.137598000	192.168.222.135	192.168.222.1	SSHv2	97 Server: Protocol (SSH-2.0-OpenSSH_9.6p1 Ubuntu-3ubuntu13.15)
74	28.143408500	192.168.222.1	192.168.222.135	SSHv2	78 Client: Protocol (SSH-2.0-paramiko_4.0.0)
75	28.143642100	192.168.222.135	192.168.222.1	TCP	60 22 → 51008 [ACK] Seq=44 Ack=25 Win=64256 Len=0
76	28.143829000	192.168.222.1	192.168.222.135	SSHv2	1310 Client: Key Exchange Init

Abb. 3 – Wireshark-Pakete

7. MITRE ATT&CK Mapping

Feld	Wert
Tactic	Credential Access (TA0006)
Technique	T1110 – Brute Force
Sub-Technique	T1110.001 – Password Guessing
Tool	Python / paramiko – selbst geschriebenes Skript
Ziel	SSH-Dienst auf Port 22 – Linux Ubuntu
Erkennung	Wazuh Rule 100300
Netzwerknachweis	Wireshark – schnelle TCP-SYN-Pakete auf Port 22

8. Was ich gelernt habe

- Wazuh erkennt SSH-Brute-Force zuverlässig durch Korrelation von Häufigkeit und Zeitfenster – ohne zusätzliche Agenten oder kommerzielle Tools.
- Die eigene Regel 100300 mit frequency="5" und timeframe="5" ist aggressiver als die eingebaute Wazuh-Standardregel und löst bei schnellen Angriffen früher aus.
- Wireshark und Wazuh ergänzen sich: Wireshark zeigt die Netzwerkebene wie TCP-Pakete und SSH-Handshake , Wazuh zeigt die Anwendungsebene wie Klartextlogs, strukturierte Alerts.
- PAM und sshd schreiben unabhängig voneinander in auth.log – das erklärt die doppelten Alerts pro Versuch.
- frequency und timeframe sind Attribute des rule-Tags in Wazuh, keine Kind-Elemente

9. Empfehlungen (Hardening)

- **SSH-Key-Authentifizierung –** Passwort-Login deaktivieren, nur noch SSH-Keys erlauben. Brute-Force gegen SSH-Keys ist praktisch nicht möglich.
- **Fail2ban –** Automatisches Sperren von IPs nach wenigen Fehlversuchen direkt auf Betriebssystem-Ebene, unabhängig von Wazuh.
- **SSH-Port ändern –** Verlagerung auf einen nicht-standardisierten Port (z.B. 2222) reduziert automatisierte Scans erheblich.
- **Active Response aktivieren –** Die Wazuh-Active-Response-Komponente kann bei Level-10-Alerts automatisch iptables-Regeln setzen und die Angreifer-IP für konfigurierbare Zeit sperren.

10. Fazit

Das Lab demonstriert erfolgreich den vollständigen Angriffs- und Detektionszyklus eines SSH-Brute-Force-Angriffs. Die Kombination aus eigenem Python-Angriffsskript, selbst konfigurierter Wazuh-Erkennungsregel und paralleler Wireshark-Analyse macht den Angriff auf drei Ebenen gleichzeitig sichtbar: Netzwerkpakete, System-Logs und SIEM-Alerts.

Besonders relevant für einen SOC-Kontext ist die Fähigkeit, nicht nur vorhandene Tools zu bedienen, sondern eigene Erkennungslogik zu entwickeln. Dieses Lab zeigt, dass effektive Angriffserkennung auch ohne kommerzielle Tools möglich ist, wenn man die zugrundeliegenden Mechanismen versteht.